

Self-Attention & Recurrent Neural Networks

1. Self-Attention

What is Self-Attention?

Self-attention is an attention mechanism used in machine learning models that weighs the importance of tokens or words in an input sequence to better understand the relationships between them[cite:4]. Rather than processing tokens in isolation, self-attention allows every token to dynamically focus on every other token in the sequence, producing rich, context-aware representations[cite:2]. It was formally introduced in the landmark 2017 paper *Attention Is All You Need*, which proposed the Transformer architecture and eliminated the need for RNNs entirely[cite:1].

How Self-Attention Works

The mechanism transforms each input token embedding into three vectors using learned weight matrices[cite:4]:

- Query (Q) — Represents what the current token is "looking for"
- Key (K) — Represents what each token "offers" or contains
- Value (V) — Represents the actual content to be aggregated

The attention score between two tokens is computed as the dot product of their Query and Key vectors, scaled by the square root of the key dimension d_k :

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

The softmax operation normalizes the scores into a probability distribution, and the final output is the weighted sum of Value vectors[cite:2]. This entire operation can be parallelized across all tokens simultaneously, giving self-attention a massive computational advantage over sequential models[cite:4].

Multi-Head Attention

In practice, the Transformer stacks multiple self-attention "heads" in parallel, each learning different aspects of token relationships[cite:3]. For example, one head might attend to syntactic dependencies (verbs to their objects), while another captures long-range semantic relationships[cite:3]. The outputs from all heads are concatenated and passed to a feedforward neural network layer, allowing the model to combine multiple relational perspectives in a single forward pass[cite:3].

Key Strengths

- **Parallelization:** All tokens are processed simultaneously, enabling efficient training on large datasets[cite:4]
- **Long-range dependencies:** Captures relationships between distant tokens with equal ease as nearby ones[cite:1]
- **Context-sensitivity:** The same word generates different representations depending on surrounding context[cite:2]
- **Interpretability:** Attention weights provide insight into which token relationships the model finds most relevant[cite:3]

Limitations

- **Quadratic complexity:** Computing attention scores for all token pairs scales as $O(n^2)$ with sequence length, making it expensive for very long sequences[cite:4]
- **No inherent positional awareness:** Unlike RNNs, self-attention has no built-in notion of order, requiring explicit positional encodings to be added to the input[cite:5]

2. Recurrent Neural Networks (RNNs)

What is an RNN?

A Recurrent Neural Network (RNN) is a class of neural network specifically designed for processing **sequential data**, where the order of elements carries meaningful information[cite:10]. Unlike standard feedforward networks that treat each input independently, RNNs introduce a **feedback loop**: the output of a hidden unit at one time step is fed back as an additional input at the next time step[cite:10]. This gives RNNs an internal memory — a hidden state — that accumulates context as the network steps through a sequence[cite:12].

How RNNs Work

At each time step t , the RNN updates its hidden state h_t using both the current input x_t and the previous hidden state h_{t-1} :

$$h_t = f(W_h \cdot h_{t-1} + W_x \cdot x_t + b)$$

where f is a non-linear activation function (commonly **tanh** or ReLU), and W_h, W_x are shared weight matrices applied at every time step[cite:8]. The key insight is **parameter sharing** — the same weights are reused at each step, keeping the model compact regardless of sequence length[cite:8].

Training uses **Backpropagation Through Time (BPTT)**, which unrolls the network across all time steps and sums the error gradients before updating weights[cite:12].

Variants of RNNs

Standard RNNs suffer from the **vanishing gradient problem**, where gradients diminish as they are propagated back through many time steps, making it hard to learn long-range dependencies. Two major variants address this:

- **Long Short-Term Memory (LSTM):** Introduces a dedicated *cell state* alongside the hidden state, regulated by input, forget, and output gates that explicitly control what information is stored, discarded, or passed forward[cite:6]
- **Gated Recurrent Unit (GRU):** A simplified version of LSTM with only two gates (reset and update), offering similar performance with fewer parameters[cite:6]

Applications

RNNs have been successfully applied to a wide range of sequential tasks[cite:10] [cite:14]:

- **Natural Language Processing (NLP):** Language modeling, sentiment analysis, text generation
- **Machine Translation:** Seq2Seq models with encoder-decoder RNN architectures
- **Speech Recognition:** Mapping audio waveform sequences to phonemes or words
- **Time Series Forecasting:** Stock price prediction, weather modeling
- **Handwriting Recognition:** Processing pen-stroke sequences

Key Limitations

- **Sequential processing:** Each time step depends on the previous one, preventing parallelization and slowing training[cite:4]
- **Vanishing/exploding gradients:** Standard RNNs struggle to retain information from distant time steps[cite:6]
- **Long-sequence bottleneck:** Even LSTMs can degrade on very long sequences, as the entire context must be compressed into a fixed-size hidden state[cite:1]

Self-Attention vs. RNNs at a Glance

Dimension	Self-Attention	RNN
Processing	Parallel (all tokens at once)	Sequential (one step at a time)
Long-range dependencies	Direct, $O(1)$ path length	Indirect, degrades with distance
Memory	Explicit attention over full context	Implicit hidden state vector
Training speed	Fast (parallelizable)	Slow (sequential dependency)
Positional awareness	Requires positional encoding	Inherently ordered by time steps
Scalability	Quadratic with sequence length	Linear with sequence length
Typical use today	Transformers, LLMs, BERT, GPT	Time series, legacy NLP, LSTM/GRU

Self-attention superseded RNNs as the dominant architecture for sequence modeling primarily because its parallel computation and direct access to all context positions result in dramatically faster training and stronger performance on long-range dependency tasks[cite:1][cite:4].